

Lab 4 Tutorial: React Context, Bottom Tab Navigator, and Drawer Navigator

This comprehensive tutorial covers essential React Native navigation and state management concepts:

1. **React Context API** - Sharing state across components without prop drilling
 2. **Bottom Tab Navigator** - Creating Instagram-style bottom tab navigation
 3. **Drawer Navigator** - Implementing side drawer menus for additional navigation
-

How to Run the Project

Follow these steps to set up and run the Lab 4 project:

Step 1: Download the Repository

Download the project from GitHub:

- Repository URL: <https://github.com/neonbranch/CPSC-357-Lab>

Step 2: Extract and Navigate

1. Extract the downloaded files
2. Navigate to the `lab4` folder
3. Then navigate to the `expo-app-v3` folder

```
cd lab4
cd expo-app-v3
```

Step 3: Install Dependencies

Install all required npm packages:

```
npm install
```

Required packages for this lab:

- `@react-navigation/native`
- `@react-navigation/bottom-tabs`
- `@react-navigation/drawer`
- `@react-navigation/native-stack`
- `react-native-gesture-handler`
- `react-native-reanimated`
- `react-native-safe-area-context`
- `react-native-screens`

Step 4: Start the Expo Development Server

Run the Expo development server:

```
npx expo start
```

Important: This project uses Drawer Navigator which requires a **Development Build**. You cannot use Expo Go for this project.

To create a development build:

```
npx expo install expo-dev-client  
npx expo run:android  
# or  
npx expo run:ios
```

Then start with:

```
npx expo start --dev-client
```

1. React Context API

React Context provides a way to share data across multiple components without passing props through every level of the component tree (prop drilling).

What is Context?

Context allows you to share data that can be considered "global" for a tree of React components, such as:

- User authentication state
- User email/username
- Theme preferences
- Shopping cart data
- User profile information

Why Use Context?

Problems with Prop Drilling:

```
// Without Context - passing props through many levels  
function App() {  
  const [userEmail, setUserEmail] = useState('');  
  return <Parent email={userEmail} setEmail={setUserEmail} />;  
}
```

```
function Parent({ email, setEmail }) {  
  return <Child email={email} setEmail={setEmail} />;  
}  
  
function Child({ email, setEmail }) {  
  return <GrandChild email={email} setEmail={setEmail} />;  
}  
  
function GrandChild({ email, setEmail }) {  
  return <Text>{email}</Text>; // Finally using the data  
}
```

Solution with Context:

```
// contexts/EmailContext.js  
import { createContext, useContext, useState } from 'react';  
  
const EmailContext = createContext();  
  
export function EmailProvider({ children }) {  
  const [email, setEmail] = useState('');  
  
  return (  
    <EmailContext.Provider value={{ email, setEmail }}>  
      {children}  
    </EmailContext.Provider>  
  );  
}  
  
export const useEmailStore = () => {  
  const context = useContext(EmailContext);  
  if (!context) {  
    throw new Error('useEmailStore must be used within EmailProvider');  
  }  
  return context;  
};
```

```
// App.js  
import { EmailProvider } from './contexts/EmailContext';  
  
function App() {  
  return (  
    <EmailProvider>  
      <Parent />  
    </EmailProvider>  
  );  
}
```

```
function Parent() {  
  return <Child />;  
}  
  
function Child() {  
  return <GrandChild />;  
}  
  
function GrandChild() {  
  const { email } = useEmailStore(); // Direct access!  
  return <Text>{email}</Text>;  
}
```

Creating a Context - Step by Step

Step 1: Create the Context File

Create a new file `contexts/EmailContext.js`:

```
import React, { createContext, useContext, useState } from 'react';  
  
const EmailContext = createContext();  
  
export function EmailProvider({ children }) {  
  const [email, setEmail] = useState('');  
  
  return (  
    <EmailContext.Provider value={{ email, setEmail }}>  
      {children}  
    </EmailContext.Provider>  
  );  
}  
  
export const useEmailStore = () => {  
  const context = useContext(EmailContext);  
  if (!context) {  
    throw new Error('useEmailStore must be used within EmailProvider');  
  }  
  return context;  
};
```

Key Points:

- `createContext()` creates a new context
- `EmailProvider` wraps components that need access to the context
- `useEmailStore` is a custom hook to access the context
- The hook throws an error if used outside the provider (safety check)

Step 2: Wrap Your App with the Provider

In `App.js`, wrap your entire app with the `EmailProvider`:

```
import { EmailProvider } from '../contexts/EmailContext';
import { NavigationContainer } from '@react-navigation/native';

export default function App() {
  return (
    <EmailProvider>
      <NavigationContainer>
        { /* Your navigation structure */ }
      </NavigationContainer>
    </EmailProvider>
  );
}
```

Step 3: Use Context in Components

Setting the email (e.g., in `LoginScreen`):

```
import { useEmailStore } from '../contexts/EmailContext';

export default function LoginScreen() {
  const { setEmail } = useEmailStore();
  const [emailInput, setEmailInput] = useState('');

  const handleLogin = () => {
    // Save email to context
    setEmail(emailInput);
    // Navigate to home
    navigation.navigate('MainTabs');
  };

  return (
    <View>
      <TextInput
        value={emailInput}
        onChangeText={setEmailInput}
        placeholder="Enter email"
      />
      <Button title="Login" onPress={handleLogin} />
    </View>
  );
}
```

Reading the email (e.g., in `ProfileScreen`):

```
import { useEmailStore } from '../contexts/EmailContext';
```

```
export default function ProfileScreen() {
  const { email } = useEmailStore();

  return (
    <View>
      <Text>Logged in as: {email}</Text>
    </View>
  );
}
```

Complete Context Example from Lab 4 Project

File: `contexts/EmailContext.js`

```
import React, { createContext, useContext, useState } from 'react';

const EmailContext = createContext();

export function EmailProvider({ children }) {
  const [email, setEmail] = useState('');

  return (
    <EmailContext.Provider value={{ email, setEmail }}>
      {children}
    </EmailContext.Provider>
  );
}

export const useEmailStore = () => {
  const context = useContext(EmailContext);
  if (!context) {
    throw new Error('useEmailStore must be used within EmailProvider');
  }
  return context;
};
```

File: `Screens/LoginScreen.js` (Setting email)

```
import { useState } from 'react';
import { View, TextInput } from 'react-native';
import { useNavigation } from '@react-navigation/native';
import { useEmailStore } from '../contexts/EmailContext';
import LoginButton from '../components/LoginButton';

export default function LoginForm() {
  const navigation = useNavigation();
  const { setEmail } = useEmailStore();
  const [emailInput, setEmailInput] = useState('');
  const [password, setPassword] = useState('');
```

```

const handleSubmit = () => {
  // Validation...
  if (!emailInput || !password) {
    Alert.alert('Error', 'Please fill in all fields');
    return;
  }

  // Save email to context
  setEmail(emailInput);
  // Navigate to MainTabs
  navigation.navigate('MainTabs');
};

return (
  <View>
    <TextInput
      value={emailInput}
      onChangeText={setEmailInput}
      placeholder="Enter your email"
    />
    <TextInput
      value={password}
      onChangeText={setPassword}
      placeholder="Enter your password"
      secureTextEntry={true}
    />
    <LoginButton onPress={handleSubmit} />
  </View>
);
}

```

Note: The `LoginButton` component uses dark green (#006400) as the theme color for consistency across the app.

File: `Screens/ProfileScreen.js` (Reading email and displaying avatar)

```

import { View, Text, StyleSheet, Image } from 'react-native';
import { useNavigation, useFocusEffect } from '@react-navigation/native';
import { SafeAreaView, SafeAreaProvider } from 'react-native-safe-area-context';
import { useEmailStore } from '../contexts/EmailContext';
import { useCallback } from 'react';

export default function ProfileScreen() {
  const navigation = useNavigation();
  const { email } = useEmailStore();

  // Close drawer when screen is focused
  useFocusEffect(
    useCallback(() => {
      navigation.closeDrawer();
    })
  );
}

```

```

    }, [navigation])
  );

  // Generate random avatar based on email
  const avatarUrl = `https://i.pravatar.cc/250?u=${email || 'default'}`;

  return (
    <SafeAreaView>
      <SafeAreaView style={styles.container}>
        <View style={styles.content}>
          <View style={styles.avatarContainer}>
            <Image
              source={{ uri: avatarUrl }}
              style={styles.avatar}
            />
          </View>
          <View style={styles.profileInfo}>
            <Text style={styles.label}>Email:</Text>
            <Text style={styles.value}>{email}</Text>
          </View>
        </View>
      </SafeAreaView>
    </SafeAreaView>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
  },
  content: {
    flex: 1,
    padding: 20,
    alignItems: 'center',
  },
  avatarContainer: {
    marginBottom: 30,
  },
  avatar: {
    width: 120,
    height: 120,
    borderRadius: 60,
    borderWidth: 3,
    borderColor: '#006400', // Dark green theme
  },
  profileInfo: {
    width: '100%',
  },
  label: {
    fontSize: 16,
    fontWeight: '600',
    marginBottom: 8,
    color: '#666',
  },
});

```



```
    },  
    value: {  
      fontSize: 18,  
      color: '#333',  
    },  
  }));
```

2. Bottom Tab Navigator

Bottom Tab Navigator provides a tab bar at the bottom of the screen, similar to Instagram, allowing users to switch between main sections of your app.

What is Bottom Tab Navigator?

Bottom Tab Navigator displays tabs at the bottom of the screen:

- Each tab represents a main section of your app
- Users can tap tabs to switch between screens
- Icons and labels are displayed for each tab
- Active tab is highlighted

Common Use Cases:

- Main app navigation (Home, Search, Profile)
- Instagram-style apps
- Apps with 3-5 main sections

Installing Bottom Tab Navigator

```
npm install @react-navigation/bottom-tabs
```

Basic Bottom Tab Navigator Setup

```
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';  
import { Ionicons } from '@expo/vector-icons';  
  
const Tab = createBottomTabNavigator();  
  
function BottomTabNavigator() {  
  return (  
    <Tab.Navigator>  
      <Tab.Screen name="Home" component={HomeScreen} />  
      <Tab.Screen name="Search" component={SearchScreen} />  
      <Tab.Screen name="Profile" component={ProfileScreen} />  
    </Tab.Navigator>  
  );  
}
```

Adding Icons to Tabs

```
import { Ionicons } from '@expo/vector-icons';

function BottomTabNavigator() {
  return (
    <Tab.Navigator
      screenOptions={({ route }) => ({
        tabBarIcon: ({ focused, color, size }) => {
          let iconName;

          if (route.name === 'Home') {
            iconName = focused ? 'home' : 'home-outline';
          } else if (route.name === 'Search') {
            iconName = focused ? 'search' : 'search-outline';
          } else if (route.name === 'Profile') {
            iconName = focused ? 'person' : 'person-outline';
          }

          return <Ionicons name={iconName} size={size} color={color} />;
        },
        tabBarActiveTintColor: '#000',
        tabBarInactiveTintColor: 'gray',
      })
    >
    <Tab.Screen name="Home" component={HomeScreen} />
    <Tab.Screen name="Search" component={SearchScreen} />
    <Tab.Screen name="Profile" component={ProfileScreen} />
  </Tab.Navigator>
);
}
```

Complete Bottom Tab Navigator Example from Lab 4

File: App.js

```
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';
import { Ionicons } from '@expo/vector-icons';
import HomeScreen from './Screens/HomeScreen';
import SearchScreen from './Screens/SearchScreen';
import AddScreen from './Screens/AddScreen';
import ActivityScreen from './Screens/ActivityScreen';
import ProfileDrawerNavigator from './ProfileDrawerNavigator';

const Tab = createBottomTabNavigator();

function BottomTabNavigator() {
  const iconMap = {
    Home: ['home', 'home-outline'],
```

```

    Search: ['search', 'search-outline'],
    Add: ['add-circle', 'add-circle-outline'],
    Activity: ['notifications', 'notifications-outline'],
    Profile: ['person', 'person-outline'],
  };

  return (
    <Tab.Navigator
      screenOptions={({ route }) => ({
        tabBarIcon: ({ focused, size, color }) => {
          const iconName = iconMap[route.name]?.[focused ? 0 : 1] || 'help-
outline';
          return <Ionicons name={iconName} size={size} color={color} />;
        },
        headerShown: false,
      })
    >
      <Tab.Screen name="Home" component={HomeScreen} />
      <Tab.Screen name="Search" component={SearchScreen} />
      <Tab.Screen name="Add" component={AddScreen} />
      <Tab.Screen name="Activity" component={ActivityScreen} />
      <Tab.Screen name="Profile" component={ProfileDrawerNavigator} />
    </Tab.Navigator>
  );
}

```

Key Features:

- **Icon Map:** Maps route names to icon pairs (filled/outline)
- **Dynamic Icons:** Shows filled icon when focused, outline when not
- **No Header:** `headerShown: false` hides the default header
- **Nested Navigator:** Profile tab uses a Drawer Navigator (see next section)

Tab Navigator Options

```

<Tab.Navigator
  screenOptions={{
    tabBarActiveTintColor: '#000',           // Color of active tab
    tabBarInactiveTintColor: 'gray',         // Color of inactive tabs
    tabBarStyle: {                           // Style the tab bar
      backgroundColor: '#fff',
      borderTopWidth: 1,
      borderTopColor: '#e0e0e0',
    },
    headerShown: false,                       // Hide header
  }}
>
  {/* Screens */}
</Tab.Navigator>

```

3. Drawer Navigator

Drawer Navigator provides a side menu that slides in from the left or right, typically used for additional navigation options like Settings, Profile, or Logout.

What is Drawer Navigator?

Drawer Navigator displays a side menu:

- Slides in from the side when opened
- Can be positioned on left or right
- Contains additional navigation options
- Often used with Bottom Tab Navigator

Common Use Cases:

- Settings menu
- Profile options
- Additional navigation items
- Logout functionality

Installing Drawer Navigator

```
npm install @react-navigation/drawer
npm install react-native-gesture-handler react-native-reanimated
```

Important: Drawer Navigator requires `react-native-reanimated` which needs a **Development Build**. It does NOT work with Expo Go.

Basic Drawer Navigator Setup

```
import { createDrawerNavigator } from '@react-navigation/drawer';

const Drawer = createDrawerNavigator();

function ProfileDrawerNavigator() {
  return (
    <Drawer.Navigator>
      <Drawer.Screen name="ProfileMain" component={ProfileScreen} />
      <Drawer.Screen name="Settings" component={SettingsScreen} />
    </Drawer.Navigator>
  );
}
```

Configuring Drawer Position

```
function ProfileDrawerNavigator() {
  return (
    <Drawer.Navigator
      screenOptions={{
        drawerPosition: 'right', // 'left' or 'right'
        headerShown: false,
        drawerStyle: { width: 250 },
      }}
    >
      <Drawer.Screen name="ProfileMain" component={ProfileScreen} />
      <Drawer.Screen name="Settings" component={SettingsScreen} />
    </Drawer.Navigator>
  );
}
```

Opening and Closing the Drawer

Opening the Drawer:

- **Swipe Gesture:** Users can swipe from the right edge of the screen to open the drawer
- **Programmatically:** Use `navigation.openDrawer()` or `navigation.dispatch(DrawerActions.openDrawer())`

Closing the Drawer:

- **Swipe Back:** Users can swipe the drawer closed or tap outside
- **Programmatically:** Use `navigation.closeDrawer()` or `navigation.dispatch(DrawerActions.closeDrawer())`
- **Auto-close:** Use `useFocusEffect` to automatically close drawer when screen is focused

Example: Auto-close Drawer on Screen Focus

```
import { useFocusEffect } from '@react-navigation/native';
import { useCallback } from 'react';

export default function ProfileScreen() {
  const navigation = useNavigation();

  // Close drawer automatically when screen is focused
  useFocusEffect(
    useCallback(() => {
      navigation.closeDrawer();
    }, [navigation])
  );

  return (
    <View>
      {/* Screen content */}
    </View>
  );
}
```

```
    );  
  }
```

Complete Drawer Navigator Example from Lab 4

File: **App.js**

```
import { createDrawerNavigator } from '@react-navigation/drawer';  
import ProfileScreen from './Screens/ProfileScreen';  
import SettingsScreen from './Screens/SettingsScreen';  
import LogoutScreen from './Screens/LogoutScreen';  
  
const Drawer = createDrawerNavigator();  
  
function ProfileDrawerNavigator() {  
  return (  
    <Drawer.Navigator  
      initialRouteName="ProfileMain"  
      screenOptions={{  
        drawerPosition: 'right',  
        headerShown: false,  
        drawerStyle: { width: 250 },  
        drawerType: 'front',  
        overlayColor: 'transparent',  
      }}  
    >  
      <Drawer.Screen  
        name="ProfileMain"  
        component={ProfileScreen}  
        options={{  
          drawerIcon: () => null,  
          title: "Profile",  
        }}  
      />  
      <Drawer.Screen name="Settings" component={SettingsScreen} />  
      <Drawer.Screen name="Logout" component={LogoutScreen} />  
    </Drawer.Navigator>  
  );  
}
```

Key Features:

- **Right Position:** `drawerPosition: 'right'` opens from the right side
- **No Header:** `headerShown: false` hides the default header
- **Custom Width:** `drawerStyle: { width: 250 }` sets drawer width
- **Multiple Screens:** Profile, Settings, and Logout screens in drawer
- **Swipe Gesture:** Users can swipe from the right edge to open the drawer

Profile Screen Example with Avatar

The Profile screen displays user information with a random avatar:

```
// Screens/ProfileScreen.js
import { View, Text, StyleSheet, Image } from 'react-native';
import { useNavigation, useFocusEffect } from '@react-navigation/native';
import { SafeAreaView, SafeAreaView } from 'react-native-safe-area-context';
import { useEmailStore } from '../contexts/EmailContext';
import Header from '../components/Header';
import { useCallback } from 'react';

export default function ProfileScreen() {
  const navigation = useNavigation();
  const { email } = useEmailStore();

  // Close drawer when screen is focused
  useFocusEffect(
    useCallback(() => {
      navigation.closeDrawer();
    }, [navigation])
  );

  // Generate random avatar based on email for consistency
  const avatarUrl = `https://i.pravatar.cc/250?u=${email || 'default'}`;

  return (
    <SafeAreaView>
      <SafeAreaView style={styles.container}>
        <View style={styles.content}>
          <View style={styles.avatarContainer}>
            <Image
              source={{ uri: avatarUrl }}
              style={styles.avatar}
            />
          </View>
          <View style={styles.profileInfo}>
            <Text style={styles.label}>Email:</Text>
            <Text style={styles.value}>{email}</Text>
          </View>
        </View>
      </SafeAreaView>
    </SafeAreaView>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: 'fff',
  },
  content: {
    flex: 1,
    padding: 20,
    alignItems: 'center',
  },
});
```

```

    },
    avatarContainer: {
      marginBottom: 30,
      alignItems: 'center',
    },
    avatar: {
      width: 120,
      height: 120,
      borderRadius: 60,
      borderWidth: 3,
      borderColor: '#006400', // Dark green theme color
    },
    profileInfo: {
      width: '100%',
      marginBottom: 30,
    },
    label: {
      fontSize: 16,
      fontWeight: '600',
      marginBottom: 8,
      color: '#666',
    },
    value: {
      fontSize: 18,
      color: '#333',
      marginBottom: 20,
    },
  },
});

```

Key Features:

- **Random Avatar:** Uses [pravatar.cc](#) service to generate avatar based on email
- **Dark Green Theme:** Avatar border uses dark green (#006400) matching app theme
- **Auto-close Drawer:** [useFocusEffect](#) ensures drawer closes when Profile screen is focused
- **Clean Layout:** Simple, centered layout with avatar and email display

4. Combining All Three: Complete Navigation Structure

This section shows how to combine React Context, Bottom Tab Navigator, and Drawer Navigator in a complete app structure.

Complete App.js Example

```

import 'react-native-gesture-handler';
import React from 'react';
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';
import { createDrawerNavigator } from '@react-navigation/drawer';
import { Ionicons } from '@expo/vector-icons';

```



```

import { EmailProvider } from './contexts/EmailContext';

// Import screens
import LoginForm from './Screens/LoginScreen';
import HomeScreen from './Screens/HomeScreen';
import ProfileScreen from './Screens/ProfileScreen';
import SearchScreen from './Screens/SearchScreen';
import AddScreen from './Screens/AddScreen';
import ActivityScreen from './Screens/ActivityScreen';
import SettingsScreen from './Screens/SettingsScreen';
import LogoutScreen from './Screens/LogoutScreen';

// Create navigators
const Stack = createNativeStackNavigator();
const Tab = createBottomTabNavigator();
const Drawer = createDrawerNavigator();

// Drawer Navigator (for Profile tab)
function ProfileDrawerNavigator() {
  return (
    <Drawer.Navigator
      initialRouteName="ProfileMain"
      screenOptions={{
        drawerPosition: 'right',
        headerShown: false,
        drawerStyle: { width: 250 },
        drawerType: 'front',
        overlayColor: 'transparent',
      }}
    >
      <Drawer.Screen
        name="ProfileMain"
        component={ProfileScreen}
        options={{
          drawerIcon: () => null,
          title: "Profile",
        }}
      />
      <Drawer.Screen name="Settings" component={SettingsScreen} />
      <Drawer.Screen name="Logout" component={LogoutScreen} />
    </Drawer.Navigator>
  );
}

// Bottom Tab Navigator
function BottomTabNavigator() {
  const iconMap = {
    Home: ['home', 'home-outline'],
    Search: ['search', 'search-outline'],
    Add: ['add-circle', 'add-circle-outline'],
    Activity: ['notifications', 'notifications-outline'],
    Profile: ['person', 'person-outline'],
  };
};

```

```

    return (
      <Tab.Navigator
        screenOptions={({ route }) => ({
          tabBarIcon: ({ focused, size, color }) => {
            const iconName = iconMap[route.name]?.[focused ? 0 : 1] || 'help-
outline';
            return <Ionicons name={iconName} size={size} color={color} />;
          },
          headerShown: false,
        })
      >
        <Tab.Screen name="Home" component={HomeScreen} />
        <Tab.Screen name="Search" component={SearchScreen} />
        <Tab.Screen name="Add" component={AddScreen} />
        <Tab.Screen name="Activity" component={ActivityScreen} />
        <Tab.Screen name="Profile" component={ProfileDrawerNavigator} />
      </Tab.Navigator>
    );
  }

// Main App Component
export default function App() {
  return (
    <EmailProvider>
      <NavigationContainer>
        <Stack.Navigator initialRouteName="Login" screenOptions={{ headerShown:
false }}>
          <Stack.Screen name="Login" component={LoginForm} />
          <Stack.Screen name="MainTabs" component={BottomTabNavigator} />
        </Stack.Navigator>
      </NavigationContainer>
    </EmailProvider>
  );
}

```

Navigation Structure

```

App (EmailProvider)
├── NavigationContainer
│   └── Stack Navigator
│       ├── Login Screen
│       └── MainTabs (Bottom Tab Navigator)
│           ├── Home Tab
│           ├── Search Tab
│           ├── Add Tab
│           ├── Activity Tab
│           └── Profile Tab (Drawer Navigator)
│               ├── ProfileMain Screen
│               ├── Settings Screen
│               └── Logout Screen

```

How Data Flows

1. **Login Screen** → Sets email in Context using `setEmail(emailInput)`
2. **All Screens** → Can access email using `const { email } = useEmailStore()`
3. **Profile Screen** → Displays email from Context and generates avatar based on email
4. **Navigation** → Bottom tabs for main navigation, Drawer for profile options
5. **Drawer Access** → Users swipe from right edge to open drawer, drawer auto-closes when Profile screen is focused

Theme Colors

The app uses **dark green (#006400)** as the primary theme color for:

- Submit buttons (Login button)
- Avatar borders
- Loading indicators
- Other primary UI elements

This creates a consistent visual identity throughout the app.

Summary

This tutorial covered essential concepts for building complex React Native applications:

- **React Context API:** Sharing state (like user email) across components without prop drilling
- **Bottom Tab Navigator:** Creating Instagram-style bottom navigation with icons
- **Drawer Navigator:** Implementing side menus for additional navigation options
- **Combined Structure:** How to use all three together in a complete app

Next Steps

- Practice creating your own Context for different data (theme, cart, etc.)
- Experiment with different tab icons and styles
- Add more screens to the Drawer Navigator
- Explore advanced navigation patterns

Lab Practice Task

Create Registration Page with Navigation to Login

Build a registration screen that links to your login page, allowing users to create an account first:

Requirements:

- **Registration Screen** with the following fields:
 - Full Name (TextInput)
 - Email (TextInput with email keyboard)
 - Username (TextInput)
 - Mobile Number (TextInput with phone keyboard)
 - Password (TextInput with secureTextEntry)

- Confirm Password (TextInput with secureTextEntry)
- Submit Button
- **Link to Login Page:** Add a button or text link on Login screen to navigate to Registration
- **Return to Login:** After successful registration, navigate back to Login screen

Form Validation: Validate all fields before submission

Validation Checklist:

- ☐ All fields must be filled (show error if any field is empty)
- ☐ Email must be valid format (contains @ and domain)
- ☐ Password and Confirm Password must match exactly
- ☐ Password must be at least 6 characters long
- ☐ Show appropriate error messages for each validation failure
- ☐ Success message appears only when all validations pass

Additional Resources

- [React Context Documentation](#)
- [React Navigation Bottom Tabs](#)
- [React Navigation Drawer](#)
- [Expo Vector Icons](#)